

Article

DKASQL: Dynamic Knowledge Adaptation for Domain-Specific Text-to-SQL

Huaxing Bian ^{1,*}, Guanrong Li ² , Yifan Wang ², Qinghan Fu ², Jian Shen ¹, Xixian Yao ¹ and Zhen Wu ²

¹ Material Branch, State Grid Jiangsu Electric Power Co., Ltd., Nanjing 211103, China; 37shenjian@163.com (J.S.); 13057591363@163.com (X.Y.)

² School of Artificial Intelligence, Nanjing University, Nanjing 210093, China; grli@smail.nju.edu.cn (G.L.); wangyifan@smail.nju.edu.cn (Y.W.); njuqhfu@gmail.com (Q.F.); wuz@nju.edu.cn (Z.W.)

* Correspondence: nuaa_bhx@sina.com

Abstract

Text2SQL aims to translate natural language queries into structured query language (SQL). LLM-based Text2SQL methods have gradually become mainstream because of their strong capabilities in language understanding and transformation. However, in real-world scenarios with non-public or limited data resources, these methods still face challenges such as insufficient domain knowledge, SQL generation that violates domain-specific constraints, and even hallucination issues. To address these challenges, this paper proposes DKASQL, a domain-specific Text2SQL method based on dynamic knowledge adaptation. The approach features an extraction module, a generation module, and an LLM-based verification module. Through an iterative “extraction–verification” and “generation–verification” mechanism, it dynamically updates the required knowledge, effectively improving both domain knowledge acquisition and the alignment between generated SQL and domain knowledge. To enhance efficiency, DKASQL also incorporates a memory storage mechanism that automatically retains commonly used domain knowledge to reduce iteration overhead. Experiments on the open-source multi-domain dataset BIRD and the ElecSQL dataset collected from the power-grid supply-chain domain show that DKASQL achieves significant performance improvements. With 7B and 32B base models, DKASQL achieves performance comparable to much larger models such as GPT-4o mini and DeepSeek-V3, with less computational overhead. For instance, on ElecSQL, DKASQL improves the execution success rate (ESR) by up to +26.9% and the result accuracy (RA) by up to +8.8% over GPT-4o mini, highlighting its effectiveness in domain-specific Text2SQL tasks.

Keywords: text-to-SQL; dynamic knowledge adaptation; large language model



Academic Editors: Carlos J. Costa and Manuela Aparicio

Received: 20 July 2025

Revised: 3 September 2025

Accepted: 12 September 2025

Published: 16 October 2025

Citation: Bian, H.; Li, G.; Wang, Y.; Fu, Q.; Shen, J.; Yao, X.; Wu, Z.

DKASQL: Dynamic Knowledge Adaptation for Domain-Specific Text-to-SQL. *Appl. Sci.* **2025**, *15*, 11121. <https://doi.org/10.3390/app152011121>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Text-to-SQL (Text2SQL) is a technology that translates natural language questions into structured query language (SQL) queries. The core objective is to simplify database access for users who lack specialized technical expertise, enabling them to retrieve and analyze structured data using plain language. With the rapid growth of data volume and complexity across many industries, Text2SQL plays an increasingly important role in connecting natural language queries with structured databases. This method is widely used in applications such as data analysis and business intelligence. For instance, in the power domain, engineers can utilize Text2SQL to access supply-chain records and assess the operational status of grid equipment. An example query is, “List the projects with the

highest failure rate in service in 2024 for the Nanjing Power Supply Company,” which supports the identification of supply-chain problems and informs the optimization of resource allocation.

Early Text2SQL systems were based on rule matching and template filling, but these approaches proved insufficient for complex queries. Subsequent developments introduced deep learning techniques, with encoder–decoder architectures [1] improving the mapping from natural language to SQL. For example, Seq2SQL breaks down SQL queries into aggregation operations, selected columns, and conditions in the WHERE clause and uses reinforcement learning to optimize the ordering of these conditions [2]. SQLNet adopts a slot-filling mechanism to explicitly capture dependencies between SELECT and WHERE clauses [3]. TypeSQL builds on this by adding type annotations, which enhance the system’s ability to recognize rare entities and numerical information [4]. In recent years, the notable capabilities of LLMs [5] have accelerated the adoption of LLM-based approaches in Text2SQL research. These methods generally fall into two categories. The first category leverages the in-context learning (ICL) capabilities of LLMs by designing prompts that guide SQL generation, exemplified by zero-shot inference with models such as ChatGPT. The second category focuses on fine-tuning open-source LLMs such as LLaMA [6] and ChatGLM [7] on standard datasets like Spider and WikiSQL to improve SQL-generation performance. Representative systems using these strategies include DB-GPT-Hub [8] and Tool-SQL [9].

Despite substantial advances in Text2SQL technology in recent years, two major challenges remain for practical deployment in specific domains. The first challenge involves a barrier posed by specialized domain knowledge. Industries such as healthcare and power systems rely heavily on technical terminology. As a result, models must possess both deep domain expertise and precise semantic understanding to accurately interpret user intent and the underlying data structures. For example, in the healthcare domain, the correct interpretation of International Classification of Diseases (ICD) codes, drug names, and diagnostic terms is essential. Similarly, in the power domain, correctly processing equipment identifiers and grid-topology information also requires substantial domain knowledge. However, high-quality training data in these fields usually needs to be annotated by domain experts, and acquiring such data is both expensive and subject to strict privacy constraints. This data scarcity limits the ability to perform domain-specific pretraining for general-purpose language models. As a result, such models often fail to capture domain-specific semantics and structural features, which may cause SQL statements to deviate from user intent and yield inaccurate query results.

The second core challenge concerns the consistency between generated SQL and domain knowledge. In real-world applications, syntactic correctness alone is insufficient. Generated SQL queries must also strictly conform to domain-specific business rules and data constraints, ensuring strong alignment with expert knowledge. Nevertheless, current LLMs often exhibit significant “domain hallucination”. They usually make plausible but inaccurate domain-specific content when producing SQL. For instance, these models may depend on generalized knowledge from their training data instead of the actual schema, leading to the introduction of nonexistent fields or table relationships or to the omission of enterprise-specific data-encoding conventions. Furthermore, even if the generated SQL is syntactically correct, the underlying query logic may conflict with actual business rules, resulting in subtle but potentially serious logical errors. For example, in the finance domain, a model might confuse the rules for calculating “account balance” and “available credit,” leading to fundamentally incorrect results.

To address the challenges, we propose Dynamic Knowledge Adaptation for Domain-Specific Text-to-SQL (DKASQL), a large language model-based approach that integrates

dynamic knowledge validation for domain-specific Text2SQL tasks. As research on domain-specific Text2SQL is still in its early stages, this paper first clarifies the task definition for domain-specific Text2SQL and then presents the DKASQL framework in detail.

The DKASQL method is composed of three primary modules: an extraction module, a generation module, and a core verification module. The extraction module extracts relevant table structures, field semantics, and domain knowledge bases based on the user's query, which helps the model understand specialized terminology and data structures. Extraction can mitigate the barrier of domain knowledge and provide a semantic foundation for SQL generation. The generation module takes the user's natural language question, along with the retrieved domain knowledge, and generates SQL queries that are both syntactically correct and consistent with business logic.

The verification module serves as a quality gatekeeper, playing a critical role in ensuring that the generated results conform to domain knowledge. It fulfills two main functions: (1) verifying the accuracy and completeness of the knowledge retrieved by the extraction module, and (2) evaluating whether the SQL output by the generation module satisfies the requirements for syntax, logic, and consistency with domain knowledge. If the verification module detects issues in extraction or generation, it analyzes the source of errors and dynamically corrects the input to the relevant modules. This forms an iterative "extraction-verification" and "generation-verification" loop, enabling dynamic adaptation and continuous refinement of domain knowledge, which ultimately enhances the accuracy and reliability of SQL generation.

In addition, a knowledge memory module is designed to store reusable knowledge identified during execution, such as typical field formats or common date-field names. This module also supports dynamic updating: after each sample is processed, it automatically identifies and records reusable domain knowledge, reducing redundant retrievals and iterations of the same information, thereby improving overall inference efficiency.

Figure 1 illustrates the DKASQL workflow. Upon receiving a natural language query, the system first engages in a "extraction-verification" loop to dynamically update the domain knowledge to be retrieved and ensure that all relevant information is available. Once knowledge extraction is complete, the system enters the "execution-verification" phase, dynamically aligning the generated SQL with domain knowledge. After processing each sample, the knowledge memory module determines whether any domain knowledge used is generalizable and, if so, adds it to the knowledge memory for future use.

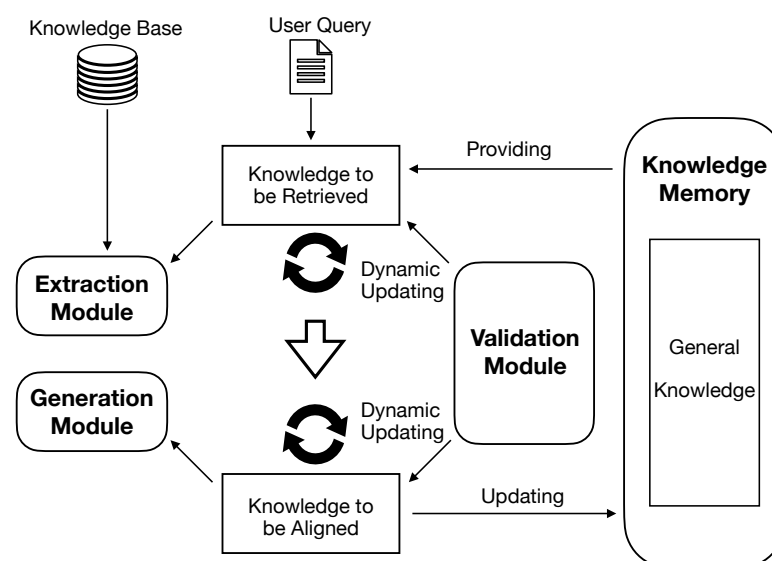


Figure 1. Workflow of DKASQL.

To validate the effectiveness of the proposed framework, we conduct experiments on the open-source multi-domain dataset BIRD [10] and the ElecSQL dataset for power-grid supply chains. Experimental results show that DKASQL effectively addresses domain-specific Text2SQL challenges, achieving state-of-the-art performance on both datasets. Notably, DKASQL attains comparable or even superior results to much larger models such as GPT-4o mini and DeepSeek-V3, even when using smaller backbone LLMs (7B, 32B parameters). For example, on the ElecSQL dataset, DKASQL improves the ESR by up to +26.9% and the RA by up to +8.8% compared to GPT-4o mini while using smaller backbone models. When built on Qwen2.5-14B, DKASQL achieves a 96.15% ESR and a 15.38% RA, surpassing GPT-4o mini (75.96% ESR, 9.62% RA) and DeepSeek-V3 (71.15% ESR, 13.46% RA). These results highlight the effectiveness of dynamic knowledge adaptation and confirm the potential of DKASQL for domain-specific Text2SQL applications.

The key contributions of our framework are summarized as follows:

- We define the task of domain-specific text-to-SQL and analyze the unique challenges it poses, including specialized terminology, data scarcity, and the need for strict alignment with domain-specific business rules.
- We propose DKASQL, a novel framework that integrates extraction, generation, and verification modules, enabling dynamic adaptation of domain knowledge during SQL generation.
- We design a knowledge memory module that automatically stores reusable domain knowledge, reducing redundant retrievals and improving overall inference efficiency.
- We conduct extensive experiments on the BIRD and ElecSQL datasets, demonstrating that DKASQL achieves state-of-the-art performance and even outperforms larger models such as GPT-4o mini and DeepSeek-V3 when using smaller backbone LLMs.

The remainder of this paper is organized as follows. Section 2 reviews related works. Section 3 presents the formal definition of the task. Section 4 describes the DKASQL framework in detail. The experimental settings and results are reported in Sections 5 and 6, respectively. Finally, Section 7 concludes this paper.

2. Related Works

Early Text2SQL research primarily relied on rule-based approaches, parsing user queries based on syntactic structures to generate corresponding SQL statements [11]. With the rise of encoder–decoder architectures, researchers began to frame the Text2SQL task as a form of neural machine translation (NMT), drawing on techniques developed in the machine translation community [12]. For example, many studies leveraged graph neural networks, sequential neural networks, and pretrained Transformer models to encode both user input and database schemas [13,14]. They then utilized decoding strategies such as slot filling [3], graph neural decoding [15], and autoregressive methods [12] to process these encoded representations and ultimately generate SQL queries.

With the widespread adoption of LLMs, the research paradigm for Text2SQL has shifted significantly. Owing to their strong performance, LLMs have surpassed earlier methods and become the foundation of current Text2SQL systems. To adapt LLMs for Text2SQL, researchers have primarily adopted two strategies: supervised fine-tuning (SFT) and prompt engineering. Supervised fine-tuning involves training relatively lightweight open-source models on datasets such as Spider and WikiSQL, which enables efficient SQL generation with modest resource requirements [16,17]. Prompt engineering, on the other hand, seeks to fully utilize the capabilities of large closed-source models (such as GPT-4 [18]) by carefully designing and optimizing prompts [19]. These models have demonstrated remarkable generalization abilities and follow input instructions with high fidelity. Initial work in prompt-based Text2SQL focused on constructing chain-of-thought (CoT) and in-

context learning (ICL) mechanisms [20]. With the improvement of LLMs, new techniques such as self-correction [21,22], self-consistency [23–25], and multi-agent collaboration [26] have been introduced, effectively mitigating output instability and further enhancing overall Text2SQL performance.

However, both supervised fine-tuning (SFT) and traditional NMT approaches rely heavily on large, high-quality labeled datasets. In real-world domain-specific scenarios, this data is rarely public or available for training. This lack of data limits the models' ability to understand unique business terminology, acronyms, and the specific data structures of a given domain. Moreover, prompt engineering with large closed-source LLMs can lead to schema hallucination, where the model invents table or column names not present in the database. This happens because the model, lacking true domain knowledge, fills in missing information based on its general training data, resulting in syntactically correct but functionally incorrect SQL queries that fail to execute. In contrast, DKASQL directly tackles these limitations with two key innovations that fundamentally change the approach to Text2SQL in domain settings. First, DKASQL introduces a dual-cycle validation process. The extraction–verification cycle proactively checks if the retrieved schema items and rules are correct and comprehensive for the user query. The generation–verification cycle then acts as a quality gate, either rejecting or repairing generated SQL queries that conflict with the established business rules and schema. DKASQL also adds a knowledge memory that accumulates reusable patterns, such as field formats and code lists. This memory acts as a dynamic cache, reducing the need for repeated context retrieval.

3. Task Formulation

To better illustrate the differences between the domain-specific Text2SQL task and the original Text2SQL task proposed in previous work [10], we first introduce the definition of the original Text2SQL task and then formalize the domain-specific Text2SQL task.

3.1. Task Formulation for Text2SQL

The input to a Text2SQL task consists of a natural language query (NLQ) and a database schema (DB schema). The natural language query Q captures the user's intent in plain text, while the database schema S defines the structure of the underlying database, including tables, columns, and relevant metadata such as column names, data types, and key constraints. The Text2SQL task can be formalized as

$$\hat{y} = \arg \max_{y \in Y_S} P(y \mid Q, S) \quad (1)$$

$$S = (T, C, F) \quad (2)$$

where Q denotes the natural language query and S represents the database schema, which is composed of a set of tables T , a set of columns C , and a set of associated metadata F (including primary keys, foreign keys, and data types). Y_S denotes the space of all valid SQL queries that are compatible with the schema S .

3.2. Task Formulation for Domain-Specific Text2SQL

As discussed in Section 1, domain knowledge is essential for models to accurately interpret user intent, domain-specific terminology, and underlying data structures. However, in real-world settings, domain knowledge is often distributed across unstructured or semi-structured resources, such as technical documentation, business-rule manuals, and expert reports. These resources are typically not systematically organized or annotated for Text2SQL tasks and often contain a significant amount of irrelevant information, making them unsuitable for direct input to the model. Furthermore, domain knowledge evolves

over time. For example, the release of new medical guidelines or updates to power-grid equipment requires continuous expansion and adaptation of the knowledge base. These dynamic requirements create challenges for knowledge integration in Text2SQL systems.

To address these challenges, we focus on domain-specific Text2SQL tasks, extending the input to include a natural language query Q , a database schema S , and an unstructured or semi-structured domain knowledge base (KB) K . The objective is to dynamically retrieve relevant knowledge entries from the KB and generate SQL queries that are both syntactically and semantically correct. This process is formalized as follows:

$$\hat{y} = \arg \max_{y \in Y_S} P(y \mid Q, S, \mathcal{R}(K, Q, S)) \quad (3)$$

where Q and S retain their definitions from the Text2SQL task in Section 3.1, K denotes the domain knowledge base, and $\mathcal{R} : K \times Q \times S \rightarrow K_{sub}$ is a domain-specific knowledge extraction function that dynamically selects the relevant subset of knowledge $K_{sub} \subseteq K$ for the current task.

4. Dynamic Knowledge Adaptation for Domain-Specific Text-to-SQL

To address the challenges of domain-specific text-to-SQL, we propose Dynamic Knowledge Adaptation for Domain-Specific Text-to-SQL (DKASQL), a dynamic knowledge adaptation framework designed to enhance both the model's comprehension of domain-specific knowledge and the execution reliability of generated SQL queries. DKASQL comprises three main modules: extraction, generation, and verification. The framework operates through iterative "extraction-verification" and "generation-verification" cycles, dynamically refining and updating domain knowledge throughout the process. Furthermore, DKASQL introduces an innovative knowledge memory mechanism to improve computational efficiency by storing and reusing generalizable domain knowledge. In the following section, we first introduce the overall structure of DKASQL and then provide details on each module.

4.1. Overall Structure

DKASQL starts with an extraction module, which gathers pertinent information—including table schemas, field semantics, and constraints—from both the database schema and the domain knowledge base. The verification module then evaluates the sufficiency of the extracted information against the knowledge memory. If deficiencies are identified, the system incrementally supplements missing knowledge until all requirements for SQL generation are fulfilled. After validating the necessary domain knowledge, the generation module synthesizes an initial SQL query based on the user's input query and the extracted information. The verification module, equipped with SQL execution capabilities, assesses the generated query for semantic correctness and consistency with business rules by executing it against the relevant database. If errors are detected, the system performs root-cause analysis, updates the knowledge base as needed, and iteratively refines the SQL query.

Upon completion of each task, DKASQL assesses whether any of the knowledge components used are generalizable. Reusable knowledge is stored in the knowledge memory module, which reduces redundant extraction and verification in future tasks and further improves overall efficiency. DKASQL provides several advantages over existing approaches. First, dynamic knowledge validation leads to higher accuracy and improved domain adaptation, while the self-correcting process decreases reliance on large amounts of annotated data. Second, by decoupling the knowledge base from the model parameters, DKASQL supports seamless updates and maintains robust performance even in low-resource settings.

Finally, the cyclical “extraction–verification–generation” process establishes a closed-loop system that continuously enhances both the quality of SQL generation and the relevance of the domain knowledge base.

4.2. Extraction Module

Unlike previous text-to-SQL approaches that mainly retrieve similar SQL examples to guide generation, DKASQL’s extraction module is tailored to identify and extract task-relevant domain knowledge, such as definitions of business metrics. The accuracy and efficiency of this module are critical, as they directly influence the quality of subsequent SQL generation.

The retrieval process consists of several key steps. First, the target knowledge for retrieval is initialized with the user’s natural language query. Next, the module conducts preliminary parsing to extract keywords and entities and determine the query intent. Using these extracted elements, the system then searches external knowledge sources to obtain relevant information. To enhance robustness, DKASQL adopts a hybrid retrieval strategy that combines vector similarity retrieval (based on BGE-m3 embeddings [27]) with keyword-based matching. Vector similarity ensures semantic alignment between queries and knowledge items, while keyword matching guarantees precise coverage of domain-specific terminology. Throughout this process, the target knowledge for retrieval is dynamically updated by the verification module to ensure alignment with the task requirements. Formally, the optimization objective of the extraction module is defined as follows:

$$\hat{k} = \{k \in K \mid \mathcal{R}(k, Q, S, \mathcal{V}^R) > \theta_R\} = \{k \in K \mid \text{sim}_{\text{emb}}(k, Q \mid S, \mathcal{V}^R) > \theta_R\} \quad (4)$$

where $\hat{k}$ represents the output of the extraction module, sim_{emb} denotes an embedding-based semantic similarity function, K is the domain knowledge base, Q is the natural language query, S is the database schema, $\mathcal{V}^R$ is the feedback from the verification module, and θ_R is the similarity threshold.

4.3. Generation Module

The generation module transforms natural language queries into executable SQL statements. This process begins with the integration of multiple inputs, including the natural language query (Q), the retrieved domain knowledge ($\hat{k}$), and the general knowledge provided by the knowledge memory module. Based on these inputs, the module generates preliminary SQL queries that comply with SQL syntax rules and the constraints imposed by the database schema (S).

The knowledge utilized in the generation process is initially derived from the database schema and domain knowledge obtained via the extraction module and is subsequently refined through feedback from the verification module. The generation process can be formally represented as follows:

$$M^n = \text{LLM}_{\text{combined}}(M^{n-1}, \mathcal{V}^{n-1}) \quad (5)$$

$$\mathcal{G}(Y^n \mid Q, \hat{k}, M, S, \mathcal{V}^G) = \prod_{t=1}^T p(y_t^n \mid y_{<t}^n, Q, \hat{k}, S, M^n, \mathcal{V}^G) \quad (6)$$

where M^n indicates the iteratively updated knowledge memory at the n -th iteration, y_t^n denotes the t -th token in the generated SQL statement Y^n , and $\mathcal{G}(Y^n \mid \cdot)$ is the optimized objective function. Prompt engineering is employed within the generation module to guide SQL construction. Specific prompt templates are designed to effectively integrate

the relevant context and constraints into the generation process. The prompt used in the generation module is as follows:

Instruction: You are an enterprise-standard SQL generator. Strictly adhere to the specifications below. Do not add any extraneous information.

Input Format:

[Table Schema]: Complete structural definitions of all relevant tables.

[Metric Name]: Name of the target metric.

[Calculation Method]: Detailed computation logic, including: relevant table names, required field names, necessary filtering conditions, complete calculation formulas, and explicit grouping dimensions (if applicable).

[Reminders]: General notes and important considerations.

Output Rules:

(a) If the calculation method contains **all** specified elements, generate a standard SQL statement.

(b) If **any** element is missing, immediately terminate and return #ERROR.

(c) Strictly follow the calculation description; do not add assumptions or supplemental information.

Output Format:

sql

SQL STATEMENT

Here, the [Reminders] correspond to the common knowledge stored in the knowledge memory module. Details of the knowledge memory module are provided in Section 4.5.

4.4. Verification Module

The verification module plays a central role in ensuring the accuracy and reliability of the text-to-SQL process by validating and evaluating the outputs of the other modules. Its core functionality consists of two primary components: extraction–verification and generation–verification. In the extraction–verification process, the module dynamically updates the knowledge to be retrieved, thereby ensuring the completeness and correctness of the information provided by the extraction module. In the generation–verification process, it dynamically aligns and audits the knowledge used during SQL generation, rigorously evaluating the quality of the SQL statements produced by the generation module. Through these mechanisms, the verification module systematically identifies potential errors and offers corrections to the domain knowledge required by the other modules, thus driving continual optimization of the entire framework.

4.4.1. Extraction–Verification Process

To assess the completeness and correctness of the information extracted by the extraction module, the verification module first acquires the database schema and relevant external knowledge retrieved from the knowledge base. It then analyzes this information against the requirements of the natural language query. The evaluation is conducted along two dimensions: completeness, which focuses on identifying missing content, and correctness, which focuses on detecting and filtering out erroneous or irrelevant information. Following this analysis, the verification module updates the knowledge to be retrieved,

enabling the extraction module to supplement missing information in subsequent iterations and avoid the recurrence of incorrect knowledge. This process is formalized as follows:

$$\mathcal{V}^R = \text{LLM}_{\text{Reflection}}(Q, K, S, \hat{k}) \quad (7)$$

where $\hat{k}$ denotes the retrieved knowledge structure, $\mathcal{V}^R$ represents the verification feedback, and Q , K , and S refer to the natural language query, the knowledge base, and the database schema, respectively.

Prompt engineering is employed within the verification module to evaluate the outputs of the retrieval process. Specific prompt templates are designed to effectively assess the completeness and correctness of the retrieved knowledge.

Instruction:

You are an enterprise-grade metric calculation engine. Execute a rigorous three-level verification process as specified below.

Input Content:

[Metric Name]: Name of the target metric.

[Calculation Method]: Complete computation logic including detailed calculation formulas, conditional judgments, and processing rules.

Processing Flow:

1. Logic Integrity Verification:

- Verify whether the calculation method is provided for the metric name, or if the calculation method is already mentioned in the metric name
- Check that all variables in the calculation formula are explicitly defined in the calculation method
- Validate that each calculation step's WHERE conditions have corresponding field support
- Confirm the existence of numerical fields corresponding to aggregate functions (SUM/COUNT, etc.)

2. Field Mapping Verification:

- Cross-check all fields mentioned in the calculation method against the **[Relevant Fields]** list
- Pay special attention to temporal fields and conditional judgment fields

Output Rules:

If no issues are detected, output the special marker #Right

If issues are detected, return the keyword

#IncompleteComponents followed by a bulleted list of the constraints and metric names identified.

4.4.2. Generation–Verification Phase

In the generation–verification phase, the verification module is equipped with SQL execution tools to rigorously assess the quality of the SQL statements generated by the generation module. The process begins with the verification module receiving candidate SQL statements and performing syntax validation to ensure consistency with the intent expressed in the user's natural language query. The module then executes the SQL statements to validate the correctness of the results and to identify any potential errors. Based on this evaluation, the verification module updates the aligned knowledge, providing targeted feedback to the generation module. The generation module subsequently refines

and regenerates the SQL statements using the updated knowledge. This quality-assurance loop is repeated until the verification module confirms that the SQL statements are free of errors. The formal representation of this process is given by

$$\mathcal{V}^G = \text{LLM}_{\text{Reflection}}(Q, K, S, Y^n) \quad (8)$$

where Y^n denotes the n -th SQL statement generated by the generation module, $\mathcal{V}^G$ represents the verification feedback on SQL quality, and Q , K , and S retain their previous definitions (natural language query, the knowledge base, and the database schema, respectively).

Prompt engineering is also employed within the verification module to facilitate systematic evaluation of generated SQL statements. Specific prompt templates are designed to support robust SQL validation throughout the iterative quality-assurance process.

Instruction:

You are an SQL expert specializing in rigorous data validation and analysis. You will be provided with a query metric, its calculation method, the corresponding SQL query, and erroneous execution results. Analyze the cause of errors and provide correction recommendations. **Do not output any SQL statements!**

Key Verification Points: 1. The SQL statement contains no placeholders (e.g., "specified date", "selected month", "selected year") - all dates are concrete values 2. Whether the SQL includes unnecessary/non-existent tables or fields 3. Whether the SQL fields exactly match those listed in the tables 4. All elements from the calculation method exist in the SQL: - Table names - Field names - Required filtering conditions - Complete calculation formulas - Explicit grouping dimensions (if applicable)

Input Format:

[Table Schema]: Complete structural definitions of all relevant tables

[Metric Name]: Target metric name

[Calculation Method]: Detailed computation logic including conditional judgments, complete formulas, and processing rules

[SQL Query]: The SQL query statement

[Execution Result]: Results from executing the SQL query

Output Rules:

List all error locations and their root causes in the following format:

- **Error Type:** [Description of discrepancy]
- **Location:** [Specific component where error occurs]
- **Cause:** [Technical explanation]
- **Recommendation:** [Corrective action]

4.5. Knowledge Memory Mechanism

In domain-specific text-to-SQL tasks, user queries frequently involve common knowledge components, such as standardized date formats in database tables or commonly used abbreviations. Repeatedly processing this shared knowledge for each query results in unnecessary computational cost and reduced system efficiency. To address this issue, we introduce a knowledge memory mechanism, which consists of a memory storage module for persistent retention of generalizable knowledge and an integrated pipeline for knowledge provision and dynamic updating.

During query processing, the system first retrieves relevant general knowledge from memory and incorporates it into the initial knowledge base to support SQL generation. After each query is answered, the verification module analyzes all utilized knowledge and classifies it as either general knowledge, which is reusable across multiple queries,

or query-specific knowledge, which is limited to the current context. Verified general knowledge is then stored in memory for future reuse. This automated and dynamic management approach effectively eliminates redundant knowledge processing and significantly improves the overall efficiency of the text-to-SQL system.

4.6. Module Collaboration Mechanism

The DKASQL framework advances text-to-SQL performance and robustness through the combined application of dynamic knowledge validation and the knowledge memory mechanism. As depicted in Figure 1 and Algorithm 1, the framework employs an iterative workflow structured around extraction–verification and generation–verification cycles. Upon receiving a natural language query, the extraction module extracts relevant metadata and domain knowledge from the database schema and external resources. The verification module then conducts completeness checks on the retrieved content. If gaps or deficiencies are identified, the knowledge to be retrieved is updated accordingly. The generation module uses the optimized context to produce preliminary SQL statements, which are subsequently evaluated by the verification module for syntactic correctness, logical alignment, and knowledge consistency. If any errors or inconsistencies are detected, the system updates the knowledge base and regenerates the SQL statement. This iterative process continues until the optimal output is achieved, ensuring both the accuracy and reliability of the generated SQL.

Algorithm 1: Dynamic knowledge-aware text-to-SQL generation

Input: Q : Natural language query;
 S : Database schema;
 K : External knowledge base
Output: $\hat{y}$: Executable SQL statement

```

/* Extraction-Verify */
repeat
     $\hat{k} \leftarrow \{k \in K \mid \mathcal{R}(k, Q, S, V^R) > \theta_R\};$  // Retrieval relative knowledge by
    extraction module
     $V^R \leftarrow \text{LLM}_{\text{Reflection}}(Q, K, S, \hat{k});$  // Verify by verification module
    if  $V^R \neq \text{"VALID"}$  then
         $K \leftarrow K \cup \text{ExpandQuery}(V^R)$ 
    ; // Update the knowledge to be retrieved
until  $V^R = \text{"VALID"}$ ;

/* Generate-Verify */
repeat
     $\mathcal{G}(Y^n \mid Q, \hat{k}, M, S, V^G) = \prod_{t=1}^T p(y_t^n \mid y_{<t}^n, Q, \hat{k}, S, M^n, V^G);$  // Generating SQL by
    generation module
     $V^G = \text{LLM}_{\text{Reflection}}(Q, K, S, Y^n);$  // Verify by verification module
    if  $V^G \neq \text{"VALID"}$  then
         $M \leftarrow M \cup \text{ExtractGeneralRules}(V^G)$ 
    ; // Update knowledge to be aligned
until  $V^G = \text{"VALID"}$ ;

/* Update Knowledge Memory */
 $M^n \leftarrow M^{n-1} \cup \text{Abstract}(Y^n, V^G);$ 

return  $\hat{y} \leftarrow Y^n$ 

```

5. Experimental Settings

5.1. Datasets

To evaluate the effectiveness of DKASQL, we conducted comprehensive experiments on two datasets: the multi-domain text-to-SQL benchmark BIRD [10] and a real-world dataset from the power-supply domain, ElecSQL. We compared DKASQL's performance against LLMs of various sizes.

The BIRD dataset is a widely used benchmark for evaluating multi-domain text-to-SQL systems. It encompasses a broad spectrum of relational database query tasks, including complex, nested, and multi-table join queries. Although BIRD originally provides query-specific annotated knowledge, such annotations are typically unavailable in real-world scenarios. To simulate a more realistic setting, we aggregated all available knowledge entries from the dataset to construct a unified domain knowledge base. As the test set annotations are not publicly available, we report model performance on the validation set, which includes 1527 natural language questions paired with corresponding SQL queries and a knowledge base of 2680 entries.

The ElecSQL dataset comprises real-world business queries collected by frontline personnel from the State Grid Corporation of China. The corresponding SQL queries were written by domain experts with deep knowledge of both power-grid operations and databases. The associated domain knowledge base was also compiled and verified by specialists. ElecSQL contains 104 question–SQL pairs and 100 curated knowledge entries.

5.2. Baselines

The DKASQL framework was designed to be adaptable to a range of mainstream LLMs. To demonstrate this flexibility, we integrated DKASQL with several representative open-source LLMs:

- Qwen series [28]: Qwen2.5-7B, Qwen2.5-14B, and Qwen2.5-Coder-7B
- DeepSeek series [29]: DeepSeek-R1-Distill-Qwen-7B and DeepSeek-R1-Distill-Qwen-32B

For comparison with commercial solutions, we also evaluated the proposed framework against two proprietary models: DeepSeek-V3 [30] and GPT-4o mini [31].

To analyze the impact of reasoning strategies, we conducted experiments under two settings:

- Original model (Orig.) [5]: Direct zero-shot inference using the base LLM without additional reasoning support.
- Chain-of-thought (CoT) [32]: Step-by-step reasoning enabled via chain-of-thought prompting.

We also compared the proposed framework with some state-of-the-art Text2SQL methods:

- DIN-SQL [21]: Decomposing text-to-SQL into smaller sub-tasks and feeding intermediate solutions into LLMs improves their reasoning ability.
- CHASE-SQL [33]: Improving text-to-SQL by combining diverse LLM generators—using divide-and-conquer decomposition, chain-of-thought reasoning with execution plans, and instance-aware synthetic example generation—with a selection agent that ranks candidates via pairwise comparisons using a fine-tuned LLM.

5.3. Metrics

To comprehensively assess model performance, we adopted two standard evaluation metrics commonly used in prior text-to-SQL studies: the execution success rate (ESR) and the result accuracy (RA) [10]. The ESR quantifies the proportion of generated SQL statements that can be successfully executed on the target database. This metric captures the model's ability to generate syntactically valid queries and accurately adhere to the database

schema. The RA measures the correctness of query outputs by comparing the results of the executed SQL statements against ground-truth answers. The RA provides a more stringent evaluation of the semantic alignment between the generated and reference queries.

5.4. Implementation Details

For experiments, all LLM services were accessed via APIs provided by the State Grid AI Platform within the secure intranet. No training or evaluation data left the internal network. All non-LLM components ran on a machine with one NVIDIA RTX 3090 GPU. These components include hybrid retrieval (BGE-m3 vector similarity + keyword matching), schema parsing, rule checking, and the SQL validator. The stack consisted of PyTorch 1.13, Transformers 4.38, FAISS 1.7 for vector indexing, and BGE-m3 for embeddings.

6. Experimental Results and Analysis

We first conducted systematic experiments on the BIRD benchmark dataset and the power-grid domain dataset, ElecSQL, covering base LLMs of varying scales (7B/32B parameter models). Additionally, to investigate the contribution of each module, we conducted modular ablation studies.

6.1. Overall Results

Tables 1 and 2 present the performance of different methods on the BIRD and ElecSQL datasets, respectively. The experimental results demonstrate that DKASQL achieved superior performance across all tested base LLMs and datasets. On the BIRD dataset, DKASQL attained an ESR of 99.28% with DeepSeek-R1-Distill-Qwen-32B, significantly outperforming the original method (63.56%). Its RA also improved from 20.66% to 35.01%. Similarly, using the same base model, DKASQL increased the ESR on the ElecSQL dataset to 18.27% (vs. 13.46% for the original method). These results strongly validate DKASQL's generalization capability and effectiveness in domain-specific Text2SQL tasks.

Furthermore, a comparison between BIRD and ElecSQL reveals that the latter presented greater challenges. Even state-of-the-art LLMs like GPT-4o mini exhibited markedly lower performance on ElecSQL, with an ESR of 75.96% and an RA of 9.62%, compared to 96.74% and 32.59% on BIRD. This suggests that while widely adopted benchmarks like BIRD effectively evaluate model performance on general Text2SQL tasks, real-world domain-specific scenarios—such as power-grid queries—involve more complex logic and specialized knowledge and demand stronger reasoning capabilities. The ElecSQL results highlight that despite LLMs' promising performance on conventional SQL-generation benchmarks, substantial improvements are still needed for practical domain-adaptive applications.

The proposed DKASQL method consistently outperformed baseline approaches on both datasets, with particularly notable gains on the more complex and domain-intensive ElecSQL dataset. For instance, when using DeepSeek-R1-Distill-Qwen-32B, DKASQL achieved a 69.46% relative improvement in RA on BIRD (20.66% → 35.01%) and a 35.74% relative improvement on ElecSQL (13.46% → 18.27%). These findings underscore DKASQL's strong advantages in handling specialized Text2SQL tasks.

Compared with state-of-the-art (SoTA) models, DKASQL still achieved superior performance on the ElecSQL dataset and comparable performance on the BIRD dataset. We attribute this difference to the varying knowledge requirements of the two benchmarks. BIRD primarily relies on general common-sense or factual knowledge that large language models can often acquire during pretraining (e.g., standard data handling practices or common entity types). In contrast, ElecSQL requires highly specialized domain knowledge, such as interpreting the “utilization rate of solidified IDs,” which cannot be accurately understood or computed without access to domain-specific expertise. This distinction

underscores the motivation behind DKASQL, which is designed to address domain-specific challenges. The relatively lower performance of SoTA models on ElecSQL further highlights the necessity of focusing on domain-specific text-to-SQL problems.

Table 1. Experimental results on the BIRD dataset. ESR and RA represent the execution success rate and the result accuracy, respectively. The best results are shown in bold.

Base Models	Methods	ESR	RA
GPT-4o mini	Orig.	96.74	32.59
DeepSeek-V3	Orig.	96.81	34.49
DIN-SQL + DeepSeek-R1-Distill-Qwen-32B	Orig.	99.13	32.22
CHASE-SQL + DeepSeek-R1-Distill-Qwen-32B	Orig.	100	35.43
DeepSeek-R1-Distill-Qwen-32B	Orig.	63.56	20.66
	CoT	94.00	31.49
	DKASQL	99.28	35.01
DeepSeek-R1-Distill-Qwen-7B	Orig.	46.15	9.97
	CoT	45.05	9.19
	DKASQL	61.08	10.04
Qwen2.5-14B	Orig.	30.51	6.58
	CoT	34.94	7.63
	DKASQL	94.91	30.51
Qwen2.5-7B	Orig.	49.35	13.82
	CoT	79.99	17.47
	DKASQL	92.44	22.16
Qwen2.5-Coder-7B	Orig.	47.39	10.56
	CoT	77.84	28.03
	DKASQL	83.18	30.57

Table 2. Experimental results on the ElecSQL dataset. ESR and RA represent the execution success rate and the result accuracy, respectively. The best results are shown in bold.

Base Models	Methods	ESR	RA
GPT-4o mini	Orig.	75.96	9.62
DeepSeek-V3	Orig.	71.15	13.46
DIN-SQL + DeepSeek-R1-Distill-Qwen-32B	Orig.	94.22	12.84
CHASE-SQL + DeepSeek-R1-Distill-Qwen-32B	Orig.	96.24	14.75
DeepSeek-R1-Distill-Qwen-32B	Orig.	69.23	13.46
	CoT	57.14	9.09
	DKASQL	95.19	18.27
DeepSeek-R1-Distill-Qwen-7B	Orig.	19.23	0.96
	CoT	11.54	0.00
	DKASQL	50.96	9.62
Qwen2.5-14B	Orig.	74.03	11.54
	CoT	60.58	9.62
	DKASQL	96.15	15.38
Qwen2.5-7B	Orig.	54.81	8.65
	CoT	57.69	11.54
	DKASQL	90.38	12.50
Qwen2.5-Coder-7B	Orig.	77.88	5.77
	CoT	70.19	7.69
	DKASQL	90.38	17.31

6.2. Ablation Study

Due to the high computational demands of models with 32B parameters, ablation experiments were performed using the more efficient 7B models. As removing the generation module would prevent the system from completing Text2SQL tasks, this analysis focused on the roles of the extraction and verification modules. Specifically, we systematically

ablated the extraction and verification modules to evaluate their individual contributions to DKASQL's overall performance. The results are shown in Tables 3 and 4.

Table 3. Ablation study on the BIRD dataset. ESR and RA represent the execution success rate and the result accuracy, respectively.

Base Model	Methods	ESR	RA
DeepSeek-R1-Distill-Qwen-32B	DKASQL	99.28	35.01
	w/o extraction	99.10	24.49
	w/o verification	72.48	25.85
Qwen2.5-Coder-7B	DKASQL	83.18	30.57
	w/o extraction	80.50	22.49
	w/o verification	47.19	19.56
Qwen2.5-7B	DKASQL	92.44	22.16
	w/o extraction	93.87	16.10
	w/o verification	67.73	18.51

Table 4. Ablation study on the ElecSQL dataset. ESR and RA represent the execution success rate and the result accuracy, respectively.

Base Model	Methods	ESR	RA
DeepSeek-R1-Distill-Qwen-32B	DKASQL	95.19	18.27
	w/o extraction	85.74	5.42
	w/o verification	75.16	11.36
Qwen2.5-Coder-7B	DKASQL	90.38	17.31
	w/o extraction	71.15	9.61
	w/o verification	83.65	14.42
Qwen2.5-7B	DKASQL	90.38	12.50
	w/o extraction	66.35	5.76
	w/o verification	63.46	10.57

The complete DKASQL framework consistently achieves the highest performance on both the BIRD and ElecSQL datasets, indicating that the interaction between the extraction and verification modules is critical for high-quality SQL generation. Excluding either module resulted in substantial performance declines.

Moreover, the ElecSQL dataset contains a larger amount of domain-specific terminology, making external knowledge extraction particularly important. When the extraction module was removed, the ESR decreased by 21.27% (from 90.38% to 71.15%), and the RA decreased by 44.48% (from 17.31% to 9.61%) when using Qwen2.5-Coder-7B. In comparison, the BIRD dataset experiences smaller declines in performance (ESR: −3.22%; RA: −26.43%), likely due to a lower reliance on specialized external knowledge.

The verification module is essential for logical validation and SQL optimization, particularly for the complex queries in BIRD. Ablating the verification module led to a 42.35% drop in the ESR (from 83.18% to 47.19%) and a 35.98% decrease in the RA (from 30.57% to 19.56%). For ElecSQL, where domain terminology (addressed by retrieval) is more important than logical refinement, the removal of the verification module resulted in a less pronounced impact (ESR: −7.45%; RA: −16.70%).

6.3. Case Studies

To provide a more intuitive analysis of DKASQL's effectiveness, we present two representative case studies in Tables 5 and 6, highlighting the roles of the extraction and verification modules. Example 1 (Table 5) contrasts the SQL-generation outcomes with and

without the extraction module, keeping all other conditions constant. In the absence of the extraction module, the model lacked access to external domain knowledge and tended to insert unnecessary or irrelevant information, resulting in SQL statements with clear deficiencies. Specifically, without the extraction module, the model only filtered records with `amount > 40`, neglecting two essential components: it failed to compute the total income (by omitting the `SUM` aggregate function) and excluded the `member_id` field while misapplying the `HAVING` clause.

Example 2 (Table 6) examines the impact of the verification module by comparing the generation results with and without this component, again under identical settings. When the verification module was removed, the model generated semantically incorrect outputs (e.g., selecting an incorrect superhero name). The verification module effectively detected such errors, offered specific suggestions for correction, and guided the model to produce the correct answer.

In Example 3 (Table 7), we present an incorrect output generated by DKASQL. The error in the SQL arose from a misinterpretation of the field name. Because the table contained multiple fields with highly similar names, DKASQL mistakenly selected a similar but incorrect field instead of the intended one.

Table 5. Example 1: Impact of the extraction module.

Component	Content
User query	Which student was able to generate income of more than USD 40?
SQL generated by DKASQL	<pre>SELECT member_id, first_name, last_name, SUM(amount) AS total_income FROM member JOIN income ON member.member_id = income.link_to_member WHERE date_received LIKE '2025-03\%' GROUP BY member_id, first_name, last_name HAVING total_income > 40;</pre>
SQL generated without extraction module	<pre>SELECT m.first_name, m.last_name FROM member m JOIN income i ON m.member_id = i.link_to_member WHERE i.amount > 40;</pre>

Table 6. Example 2: Impact of verification module.

Component	Content
User query	Who is the publisher of Sauron?
SQL generated by DKASQL	<pre>SELECT T2.publisher_name FROM superhero AS T1 INNER JOIN publisher AS T2 ON T1.publisher_id = T2.id WHERE T1.full_name = 'Maya Mclean';</pre>
SQL generated without verification module	<pre>SELECT publisher_name FROM superhero JOIN publisher ON superhe-ro.publisher_id = publisher.id WHERE superhero_name = 'Sauron';</pre>

Table 7. Example 3: A case from the ElecSQL dataset. The red highlights in the SQL generated by DKASQL indicate the incorrect components.

Component	Content
User query	Please provide the Planned Reporting Status by Project Unit for this year.
Knowledge	Planned Reporting Status by Project Unit Take the Service Demand Plan Log Table (MMPL_D007) and apply the following filters: * Non-Material Demand Plan Workflow Number field is not empty. * Approval Status field is "Provincial - Approved." * Historical Data field is empty. * Plan Batch Name contains the keyword "Service." * Approval Time field falls within the selected time range. Statistics by project unit: * Number of Batches: Count of distinct "Plan Batches." * Number of Entries: Count of filtered entries. * Amount: Sum of (Purchase Request Price × Purchase Request Quantity) from the filtered data. Preconditions: * Non-Material Demand Plan Workflow Number field is not empty. * Approval Status field is "Provincial - Approved." * Historical Data field is empty. * Plan Batch Name contains the keyword "Service." Statistical Scope: * Aggregate the full dataset by project unit: * Number of Batches: Count of distinct "Plan Batches." * Number of Entries: Count of filtered entries. Amount: Sum of (Purchase Request Price × Purchase Request Quantity).
SQL generated by DKASQL	<pre> WITH TEMP AS (SELECT B.ZCITY_TXT, C.PREIS, C.MENGE, A.ZZBJHBH FROM "MMPL_D007" A JOIN "MMIO_D008" B ON SUBSTRING(A.WERKS, 3, 1) = B.NACE JOIN "MMPU_D001" C ON A.CGSQH = C.BANFN AND A.CGSQHXMH = C.BNFPO WHERE A.ZWFHZ_STATE = 'Provincial - Approved' AND A.ZWF_ID IS NOT NULL AND A.ZWF_ID <> '' AND (A.ZHISTORY IS NULL OR A.ZHISTORY = '') AND A.ZZBJHMC LIKE '%service%' AND A.ZPROVINCE_PASS_D BETWEEN 'ym1'AND'2024' AND '2025') SELECT ZCITY_TXT AS Project Unit, COUNT(DISTINCT ZZBJHBH) AS Number of Batches, COUNT(*) AS Number of Entries, CAST(SUM(PREIS * MENGE)) AS Price FROM TEMP GROUP BY ZCITY_TXT ORDER BY ZCITY_TXT; </pre>

6.4. Impact of Knowledge Memory on Framework

To visually illustrate the function of the knowledge memory module, we present examples from the ElecSQL power-grid dataset in Table 8. Due to space limitations, three representative knowledge entries stored in the knowledge memory module are shown, along with one example of domain-specific knowledge not included in the module for comparison. The general knowledge entries primarily capture foundational elements common across multiple samples, such as date format conventions (e.g., the “YYYYMMDD” format) and regional coding standards (e.g., field names and value specifications used across different regions). In contrast, domain-specific knowledge addresses computational logic relevant to particular business scenarios. This distribution highlights the intended design of the knowledge memory module: consolidating general knowledge to minimize redundant computation while maintaining the flexibility to handle specialized cases. As a result, the module significantly improves the overall efficiency of the DKASQL framework.

Table 8. Examples illustrating the knowledge memory module.

Component	Content
Examples of knowledge stored in knowledge memory module	1. The specific date format is "YYYYMMDD". 2. The field name for the year follows the format "YYYY" without month or day. 3. For city name information, the values must strictly adhere to the following rules: City names must use the official abbreviated form of prefecture-level or higher administrative units, consisting solely of Chinese characters without any administrative suffixes (e.g., city, district) or special characters.
Knowledge for specific query	Fixed ID Utilization Rate: $A1 = \sum (\text{Total Winning Bid Price})$ $B1 = \sum (\text{Total Winning Bid Price where the Fixed ID starts with "9", "52", "10GH", or "G00L"})$ $A2 = \sum (\text{Total Purchase Order Value, including tax})$ $B2 = \sum (\text{Total Purchase Order Value, including tax, where the Fixed ID starts with "9", "52", "10GH", or "G00L"})$ $\text{Fixed ID Utilization Rate} = (B1 + B2) / (A1 + A2) \times 100\%$

7. Conclusions

We present DKASQL, a domain-specific text-to-SQL approach grounded in a dynamic knowledge validation mechanism. By leveraging iterative dynamic knowledge optimization, DKASQL effectively addresses major challenges in domain-specific text-to-SQL tasks, such as domain knowledge limitations and execution reliability. The framework implements collaborative “extraction–verification” and “generation–verification” cycles through the coordinated operation of extraction, generation, and evaluation modules, thereby enabling continuous updating and refinement of domain knowledge. The integrated knowledge memory module further supports the persistent accumulation and reuse of domain-specific knowledge.

Experimental results show that DKASQL achieves higher accuracy than state-of-the-art LLMs, including GPT-4o mini and DeepSeek-V3, on both the BIRD benchmark dataset and the ElecSQL power-grid dataset, while utilizing only 32 billion parameters. Remarkably, even with a 7-billion-parameter base model, DKASQL attains performance on par with substantially larger models. Consistent improvements across diverse datasets and model backbones demonstrate the generalizability and robustness of the proposed approach. This collaborative paradigm not only highlights the potential for achieving strong performance

with compact models through effective cooperation but also provides extensible interfaces for dynamic knowledge updates owing to its modular architecture.

Limitations

First, DKASQL depends on the quality and recall of the extraction module: incomplete or noisy retrieval can misguide both generation and verification, especially when schemas are implicit or poorly documented. Second, iterative extraction–verification and generation–verification loops add latency and computational cost, which can be noticeable in interactive use or with very large schemas. Third, the knowledge memory can bias the system toward past solutions. If schemas or rules change, stale patterns may degrade accuracy. The current work does not implement automatic decay, version pinning, or anomaly detection for memory entries.

Future work

We will jointly learn extraction and verification using execution feedback, with contrastive objectives that penalize missing fields or incorrect joins and reward stronger schema–query alignment prior to decoding. We will introduce versioned and private knowledge memory with tenant-level namespacing, schema-linked version tags, time-based decay, detectors for sensitive tokens, and audit tooling. Finally, we will add a planner based on an intermediate representation that maps a question to a compositional operator graph prior to SQL rendering; the validator will check each step against the schema and business rules to reduce hallucinated joins and missing filters.

In summary, DKASQL advances domain-specific text-to-SQL through dynamic knowledge validation and coordinated modules, but its reliability still depends on high-recall retrieval, efficient iterative control, and well-managed memory. Addressing these points will further improve accuracy, responsiveness, and safety in real deployments.

Author Contributions: Conceptualization, H.B., G.L., and Y.W.; methodology, H.B., G.L., and Y.W.; software, G.L. and Y.W.; validation, H.B. and Q.F.; investigation, H.B., G.L., Y.W., Q.F., and Z.W.; resources, H.B., J.S., and X.Y.; data processing, Y.W., Q.F., J.S., and X.Y.; writing—original draft preparation, G.L. and Y.W.; writing—review and editing, H.B., Q.F., J.S., X.Y., and Z.W.; supervision, H.B., and Z.W.; project administration, H.B.; funding acquisition, H.B., and Z.W. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by the Science and Technology Project of State Grid Jiangsu Electric Power Co., Ltd. (J2024085).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: We utilize two datasets: BIRD [10] and ElecSQL. BIRD is an open-access dataset available at <https://bird-bench.github.io>. ElecSQL is a private dataset that we provide at <https://drive.google.com/drive/folders/1cXU4Y8M3g3tdXWzxYCEPE9VQvh8J4DDh?usp=sharing> (accessed on 11 September 2025).

Acknowledgments: We gratefully acknowledge Weihao Chen for his insightful discussions and valuable assistance with data processing, which contributed significantly to the quality of this work.

Conflicts of Interest: Some of the authors are employees of the Material Branch, State Grid Jiangsu Electric Power Ltd., which also funded this research. The authors declare that this affiliation did not influence the objectivity of the research. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

DKASQL	Dynamic Knowledge Adaptation for Domain-Specific Text-to-SQL
LLM	large language model
Text2SQL	text-to-SQL
SQL	structured query language
ESR	execution success rate
RA	result accuracy
NMT	neural machine translation
SFT	supervised fine-tuning
CoT	chain-of-thought
ICL	in-context learning
SoTA	state of the art

References

1. Sutskever, I.; Vinyals, O.; Le, Q.V. Sequence to sequence learning with neural networks. In Proceedings of the 28th International Conference on Neural Information Processing Systems—Volume 2, Montreal, QC, Canada, 8–13 December 2014; NIPS’14; MIT Press: Cambridge, MA, USA, 2014; pp. 3104–3112.
2. Zhong, V.; Xiong, C.; Socher, R. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv* **2017**, arXiv:1709.00103. [\[CrossRef\]](#)
3. Xu, X.; Liu, C.; Song, D. Sqlnet: Generating structured queries from natural language without reinforcement learning. *arXiv* **2017**, arXiv:1711.04436. [\[CrossRef\]](#)
4. Yu, T.; Li, Z.; Zhang, Z.; Zhang, R.; Radev, D. TypeSQL: Knowledge-Based Type-Aware Neural Text-to-SQL Generation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*; Association for Computational Linguistics: New Orleans, LA, USA, 2018; pp. 588–594.
5. Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language models are few-shot learners. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 1877–1901.
6. Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. Llama: Open and efficient foundation language models. *arXiv* **2023**, arXiv:2302.13971. [\[CrossRef\]](#)
7. Du, Z.; Qian, Y.; Liu, X.; Ding, M.; Qiu, J.; Yang, Z.; Tang, J. Gln: General language model pretraining with autoregressive blank infilling. *arXiv* **2021**, arXiv:2103.10360. [\[CrossRef\]](#)
8. Zhou, F.; Xue, S.; Qi, D.; Shi, W.; Zhao, W.; Wei, G.; Zhang, H.; Jiang, C.; Jiang, G.; Chu, Z.; et al. DB-GPT-Hub: Towards Open Benchmarking Text-to-SQL Empowered by Large Language Models. *arXiv* **2024**, arXiv:2406.11434. [\[CrossRef\]](#)
9. Wang, Z.; Zhang, R.; Nie, Z.; Kim, J. Tool-assisted agent on sql inspection and refinement in real-world scenarios. *arXiv* **2024**, arXiv:2408.16991. [\[CrossRef\]](#)
10. Li, J.; Hui, B.; Qu, G.; Yang, J.; Li, B.; Li, B.; Wang, B.; Qin, B.; Geng, R.; Huo, N.; et al. Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 42330–42357.
11. Iyer, S.; Konstas, I.; Cheung, A.; Krishnamurthy, J.; Zettlemoyer, L. Learning a Neural Semantic Parser from User Feedback. In Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics 2017, Vancouver, BC, Canada, 30 July–4 August 2017.
12. Wang, B.; Shin, R.; Liu, X.; Polozov, O.; Richardson, M. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 7567–7578.
13. Hwang, W.; Yim, J.; Park, S.; Seo, M. A comprehensive exploration on wikisql with table-aware word contextualization. *arXiv* **2019**, arXiv:1902.01069. [\[CrossRef\]](#)
14. Cao, R.; Chen, L.; Chen, Z.; Zhao, Y.; Zhu, S.; Yu, K. LGESQL: Line Graph Enhanced Text-to-SQL Model with Mixed Local and Non-Local Relations. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*; Association for Computational Linguistics: New Orleans, LA, USA, 2021; pp. 2541–2555.
15. Zhan, Y.; Sun, Y.; Weng, H.; Cui, L.; Wang, G.; Xie, J.; Tian, Y.; Yin, X.; Liu, B.; Huang, D. FuncEvalGMN: Evaluating Functional Correctness of SQL via Graph Matching Network. *arXiv* **2024**, arXiv:2407.14530. [\[CrossRef\]](#)
16. Choi, D.; Shin, M.C.; Kim, E.; Shin, D.R. Ryansql: Recursively applying sketch-based slot fillings for complex text-to-sql in cross-domain databases. *Comput. Linguist.* **2021**, *47*, 309–332. [\[CrossRef\]](#)

17. Maamari, K.; Abubaker, F.; Jaroslawicz, D.; Mhedhbi, A. The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models. In Proceedings of the NeurIPS 2024 Third Table Representation Learning Workshop, Vancouver, BC, Canada, 14 December 2024.
18. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. Gpt-4 technical report. *arXiv* **2023**, arXiv:2303.08774. [[CrossRef](#)]
19. Li, H.; Zhang, J.; Liu, H.; Fan, J.; Zhang, X.; Zhu, J.; Wei, R.; Pan, H.; Li, C.; Chen, H. Codes: Towards building open-source language models for text-to-sql. *Proc. ACM Manag. Data* **2024**, *2*, 1–28. [[CrossRef](#)]
20. Dong, X.; Zhang, C.; Ge, Y.; Mao, Y.; Gao, Y.; Lin, J.; Lou, D. C3: Zero-shot text-to-sql with chatgpt. *arXiv* **2023**, arXiv:2307.07306. [[CrossRef](#)]
21. Pourreza, M.; Rafiei, D. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 36339–36348.
22. Chen, X.; Lin, M.; Schaerli, N.; Zhou, D. Teaching Large Language Models to Self-Debug. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, Toronto, ON, Canada, 9–14 July 2023.
23. Thorpe, D.G.; Duberstein, A.J.; Kinsey, I.A. Dubo-SQL: Diverse Retrieval-Augmented Generation and Fine Tuning for Text-to-SQL. *arXiv* **2024**, arXiv:2404.12560. [[CrossRef](#)]
24. Talaei, S.; Pourreza, M.; Chang, Y.C.; Mirhoseini, A.; Saberi, A. Chess: Contextual harnessing for efficient sql synthesis. *arXiv* **2024**, arXiv:2405.16755. [[CrossRef](#)]
25. Lee, D.; Park, C.; Kim, J.; Park, H. MCS-SQL: Leveraging Multiple Prompts and Multiple-Choice Selection For Text-to-SQL Generation. In Proceedings of the 31st International Conference on Computational Linguistics, Abu Dhabi, United Arab Emirates, 19–24 January 2025; pp. 337–353.
26. Wang, B.; Ren, C.; Yang, J.; Liang, X.; Bai, J.; Chai, L.; Yan, Z.; Zhang, Q.W.; Yin, D.; Sun, X.; et al. MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL. In Proceedings of the 31st International Conference on Computational Linguistics, Abu Dhabi, United Arab Emirates, 19–24 January 2025; pp. 540–557.
27. Chen, J.; Xiao, S.; Zhang, P.; Luo, K.; Lian, D.; Liu, Z. BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation. *arXiv* **2024**, arXiv:2402.03216. [[CrossRef](#)]
28. Team, Q. Qwen2 technical report. *arXiv* **2024**, arXiv:2412.15115.
29. Guo, D.; Yang, D.; Zhang, H.; Song, J.; Zhang, R.; Xu, R.; Zhu, Q.; Ma, S.; Wang, P.; Bi, X.; et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv* **2025**, arXiv:2501.12948.
30. Liu, A.; Feng, B.; Xue, B.; Wang, B.; Wu, B.; Lu, C.; Zhao, C.; Deng, C.; Zhang, C.; Ruan, C.; et al. Deepseek-v3 technical report. *arXiv* **2024**, arXiv:2412.19437.
31. Hurst, A.; Lerer, A.; Goucher, A.P.; Perelman, A.; Ramesh, A.; Clark, A.; Ostrow, A.; Welihinda, A.; Hayes, A.; Radford, A.; et al. Gpt-4o system card. *arXiv* **2024**, arXiv:2410.21276. [[CrossRef](#)]
32. Kojima, T.; Gu, S.S.; Reid, M.; Matsuo, Y.; Iwasawa, Y. Large language models are zero-shot reasoners. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 22199–22213.
33. Pourreza, M.; Li, H.; Sun, R.; Chung, Y.; Talaei, S.; Kakkar, G.T.; Gan, Y.; Saberi, A.; Ozcan, F.; Arik, S.O. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *arXiv* **2024**, arXiv:2410.01943.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.